

IV: Les SVM, les kernels et les k plus proches voisins

Guillaume Guex guillaume.guex@unil.ch

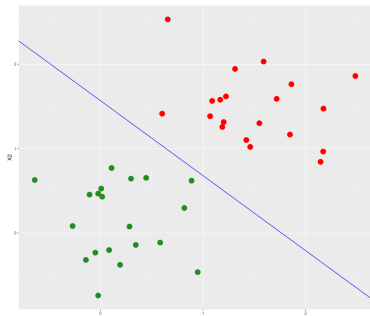
Support Vector Machines (SVM)

Les **Support Vector Machines (SVM)** (en français Machines à Vecteurs de Support) sont une famille de méthodes de Machine Learning servant principalement à la classification.

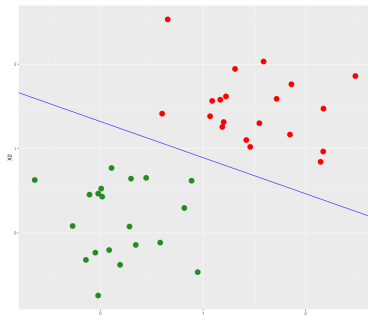
- Ce sont des **modèles linéaires**, développés par Valdimir N. Vapnik et Alexey Chervonenkis en 1963.
- Les **fonction de kernels** (noyaux), créées en 1995 par le même groupe de recherche, permettent d'appliquer ces méthodes aux données non-linéaires.
- Sont grandement utilisés dans durant les décennies 1990 et 2000, avant d'être détrônés par les **réseaux de neurones** au début des années 2010s.
- Existent en version **régression**, appelée **Support Vector Regression (SVR)** (que nous ne verrons pas).

La frontière de décision

Revenons sur la classification binaire. Avec la régression linéaire et logistique, on obtient une **frontière de décision** :



Régression linéaire



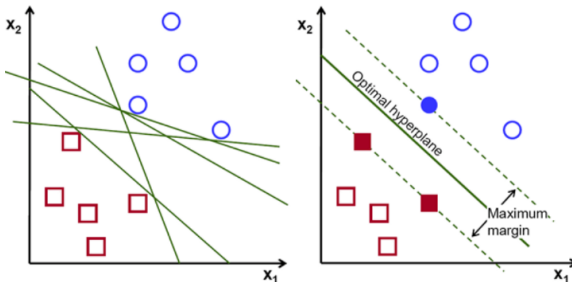
Régression logistique

Peut-on construire une méthode qui trace une

« **bonne** » **frontière** ?

Les vecteurs de support

On peut essayer de trouver une frontière qui **maximise la marge** entre les deux groupes de point. Cette frontière va être déterminée uniquement par certains points : les **vecteurs de support**



Source : <https://towardsdatascience.com/support-vector-machine-introduction-to-machine-learning-algorithms-934a444fca47>

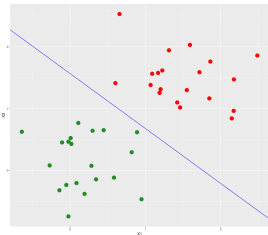
Ce principe est la base des algorithmes appelés **Support Vector Machines (SVM)**.

Pour trouver cette frontière, on doit utiliser des labels $y_i = 1$ pour un groupe et $y_i = -1$ pour l'autre groupe, et résoudre le **problème avec contraintes** suivant

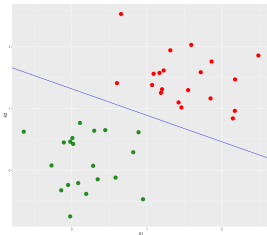
$$\begin{aligned} \min_{w_0, w_1, \dots, w_p} \quad & w_1^2 + \dots + w_p^2 \\ \text{sous contraintes} \quad & y_i(w_0 + w_1 x_{i1} + \dots + w_p x_{ip}) \geq 1 \quad \forall i \end{aligned}$$

où les **paramètres** $w_0, w_1, \dots, w_p$ vont définir la frontière de décision $w_0 + w_1 x_1 + \dots + w_p x_p = 0$.

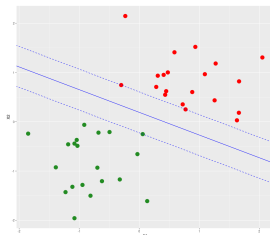
Support Vector Machines



Régression linéaire

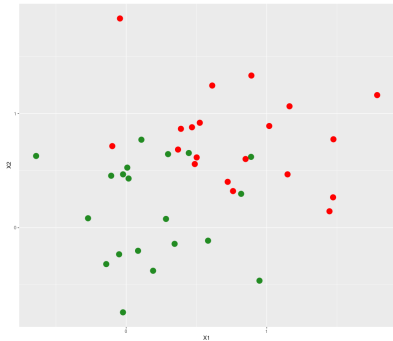
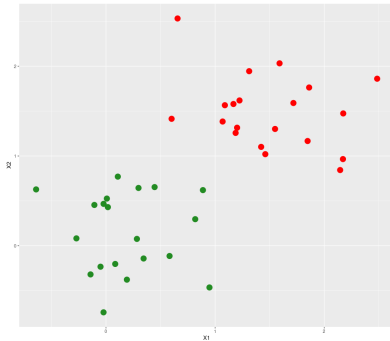


Régression logistique



SVM

En réalité, la plupart des données ne contiennent pas deux groupes **séparables par un hyperplan**



On peut résoudre ce problème en ajoutant des **pénalités** sur les points qui passent la frontière. La **fonction de perte** devient

$$L(\mathbf{w}) = w_1^2 + \dots + w_p^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w_0 + w_1 x_{i1} + \dots + w_p x_{ip}))$$

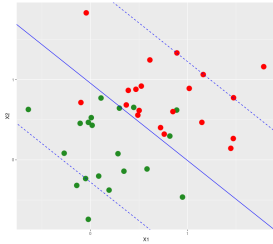
où $C > 0$ est un **hyperparamètre** qui met plus de pénalité sur les points dépassant la frontière.

Le terme $\max(0, 1 - y_i(w_0 + w_1 x_{i1} + \dots + w_p x_{ip}))$ s'appelle le **hinge loss**.

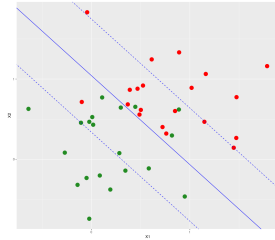
On trouve la solution avec une méthode de descente du gradient.

Support Vector Machines

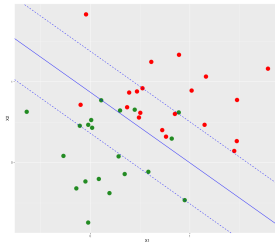
En augmentant C , le nombre de **vecteurs de support** diminue



$C = 0.1$



$C = 0.5$

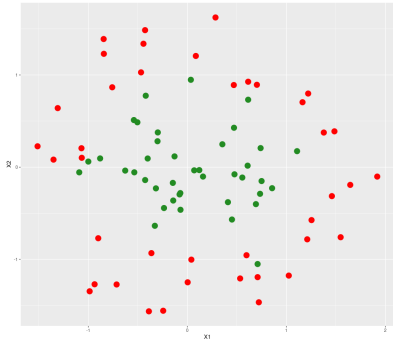
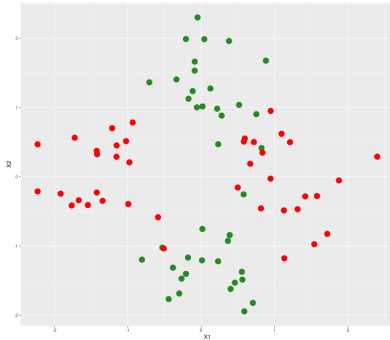


$C = 1$

Les kernels

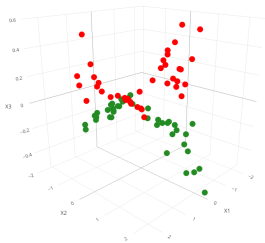
Les frontières non-linéaires

La plupart du temps, nous avons des données qui ne suivent pas une **frontière linéaire**

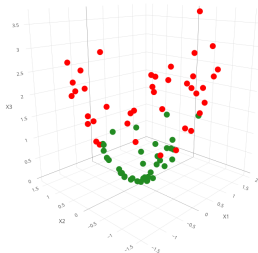


Features expansion

Cependant, en projetant les points dans un **espace de dimension supérieure**, ces données peuvent être séparées par un hyperplan



$$X_3 = X_1^2 - X_2^2$$



$$X_3 = X_1^2 + X_2^2$$

C'est le principe de la **features expansion** : on transforme les données $\mathbf{x}_1, \dots, \mathbf{x}_n$ en de nouvelles données $\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_n)$ grâce à une **fonction de base** $\phi(\cdot)$.

En réalité, de nombreuses méthodes de Machine Learning, à la place d'utiliser la **matrice des données** dans l'entraînement

$$\mathbf{X} = \begin{pmatrix} \text{—} & \mathbf{x}_1^\top & \text{—} \\ \text{—} & \mathbf{x}_2^\top & \text{—} \\ & \vdots & \\ \text{—} & \mathbf{x}_n^\top & \text{—} \end{pmatrix},$$

peuvent utiliser la **matrice des produits scalaires** ou **kernel**

$$\mathbf{K} = \mathbf{X}\mathbf{X}^\top = \begin{pmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 & \cdots & \mathbf{x}_1^\top \mathbf{x}_n \\ \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 & \cdots & \mathbf{x}_2^\top \mathbf{x}_n \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_n^\top \mathbf{x}_1 & \mathbf{x}_n^\top \mathbf{x}_2 & \cdots & \mathbf{x}_n^\top \mathbf{x}_n \end{pmatrix}$$

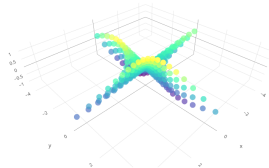
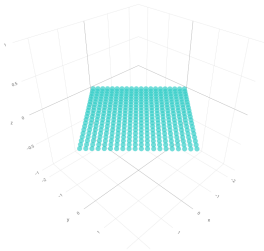
Le **kernel trick** consiste à remplacer tous les produits scalaires $\mathbf{x}_i^\top \mathbf{x}_j$ par une **fonction kernel** $k(\mathbf{x}_i, \mathbf{x}_j)$ particulière. Cela aura pour effet de **transformer l'espace des points**.

Les fonctions kernel les plus utilisées sont :

- Le kernel **polynomial** $k(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^\top \mathbf{x}_j + c)^d$
- La **radial basis function** $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$
- Le kernel **sigmoid** $k(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i^\top \mathbf{x}_j + c)$

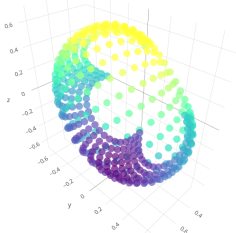
où γ, c, d sont des **hyperparamètres**.

Les fonctions kernel

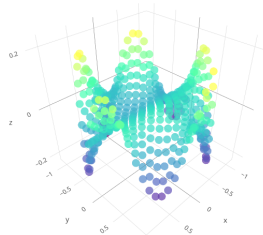


$$k(\mathbf{x}_i, \mathbf{x}_j) = \left(\frac{1}{2}\mathbf{x}_i^\top \mathbf{x}_j\right)^3$$

Situation initiale



$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{1}{2}(\mathbf{x}_i - \mathbf{x}_j)^2\right)$$



$$k(\mathbf{x}_i, \mathbf{x}_j) = \tanh\left(\frac{1}{2}\mathbf{x}_i^\top \mathbf{x}_j\right)$$

Pour une visualisation des effets des différents **hyperparamètres**,
voir

<https://towardsdatascience.com/visualizing-the-effect-of-hyperparameters-on-support-vector-machines-b9eef6f7357b>

Classes multiples

Les SVMs est un **classifieur binaire**, il ne s'applique qu'avec une variable réponse qui a **deux modalités**. Pour pouvoir appliquer un classifieur binaire à une variable avec $m > 2$, on peut utiliser les stratégies suivantes :

- Le **one-vs-rest** : on entraîne m classifieurs binaires, un par modalité, en considérant toutes les autres modalités comme résultats « négatifs ».
- Le **one-vs-one** : on entraîne $m(m - 1)/2$ classifieurs binaires, entre chaque pair de modalités.

La stratégie **one-vs-rest** est la plus utilisée :

- Elle est généralement plus **rapide** (m classifieurs vs $m(m-1)/2$).
- Chaque classifieur peut est **interprété** pour avoir de l'information sur sa modalité.
- Les zones ambiguës doivent être réglées en comparant un **score de confiance** donné par chaque classifieur (p.ex., en comparant $\mathbb{P}_1(Y=1)$ vs $\mathbb{P}_2(Y=2)$).
- Elle peut servir pour de la classification **multi-labels**.

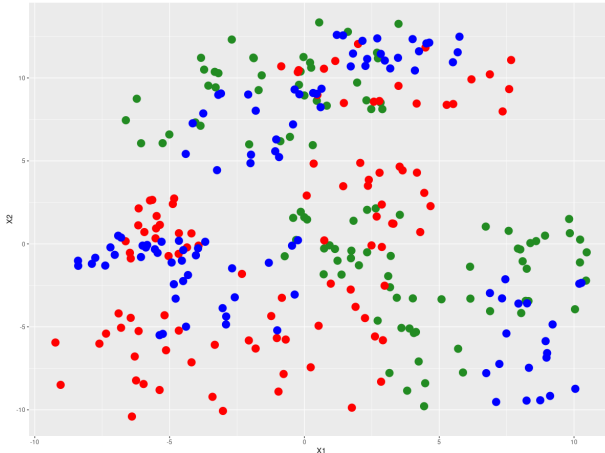
La stratégie **one-vs-one** est préférée dans les cas où l'algorithme ne **scale** pas bien (sensible à l'effet d'échelle) :

- Si le plus petit nombre de données utilisé pour chaque modèle (on entraîne que sur 2 classes à chaque fois) accélère fortement l'apprentissage, elle sera plus **rapide**.
- Chaque point reçoit un **vote** pour chaque classifieur, et la classe majoritaire est sélectionnée.
- En cas d'**égalité**, on peut aller regarder l'**agrégation des scores de confiances** de chaque classifieur.

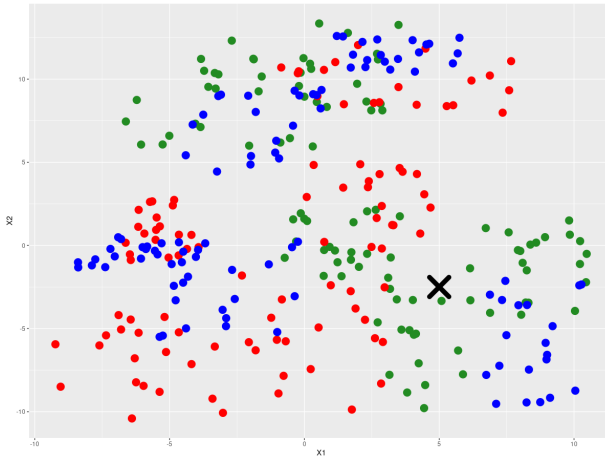
Les k plus proches voisins (kNN)

Les données peu structurées

Pour le moment, nous n'avons vu que des méthodes supposant un **modèle linéaire** entre les données ou les données transformées. Cependant, il existe des données où la structure est très irrégulière



Malgré cela, visuellement, on pourrait prédire intuitivement la position d'un **nouveau point**



Les k plus proches voisins (k NN)

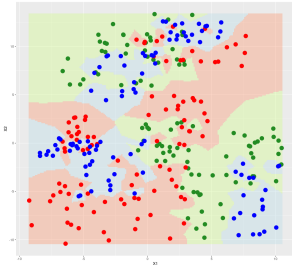
L'algorithme des k plus proches voisins, en anglais k -nearest neighbors (k -NN), est un algorithme **non-paramétrique, supervisé, de classification**.

Le principe est simple. Il n'y a pas d'entraînement à proprement parler. Pour prédire un nouvel individu, on va se baser sur le **jeu d'entraînement** :

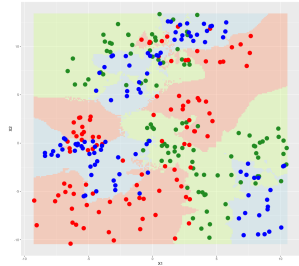
- On regarde les k plus proches voisins du nouveau point, par rapport à une certaine **distance**.
- On attribue la classe de ce nouveau point à la **classe majoritaire** trouvée chez les voisins.
- En cas d'égalité, on peut trancher en fonction de la distance.

L'**hyperparamètre k contrôle la complexité** (la complexité diminue avec k).

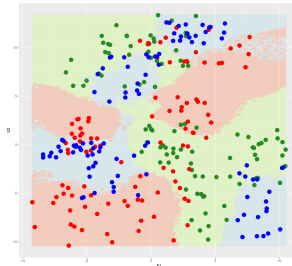
Les k plus proches voisins (k NN)



$k = 1$



$k = 5$



$k = 20$